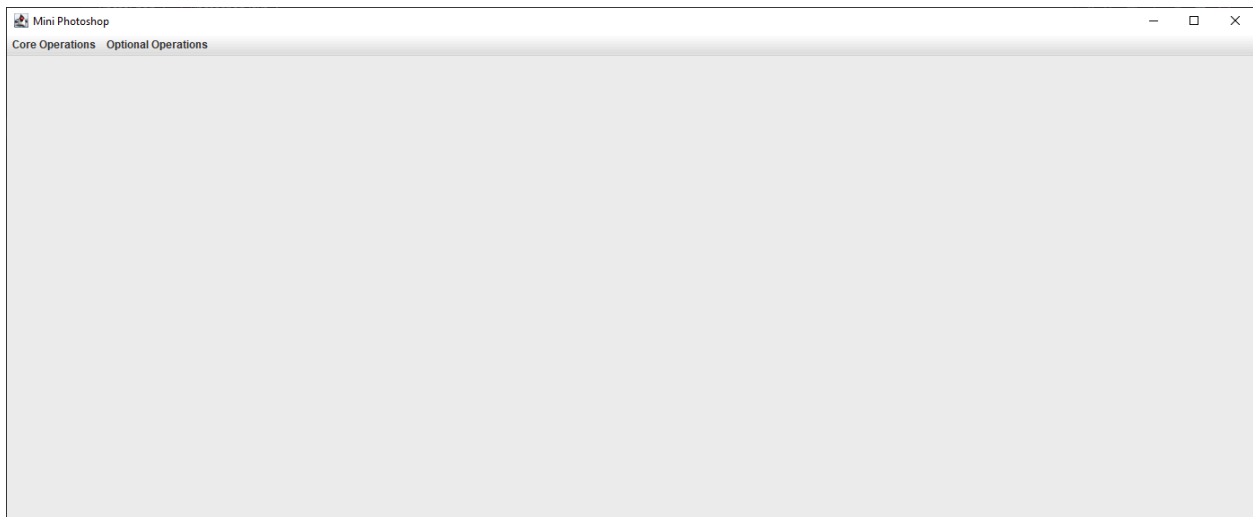


CMPT 365 Project 2 Report

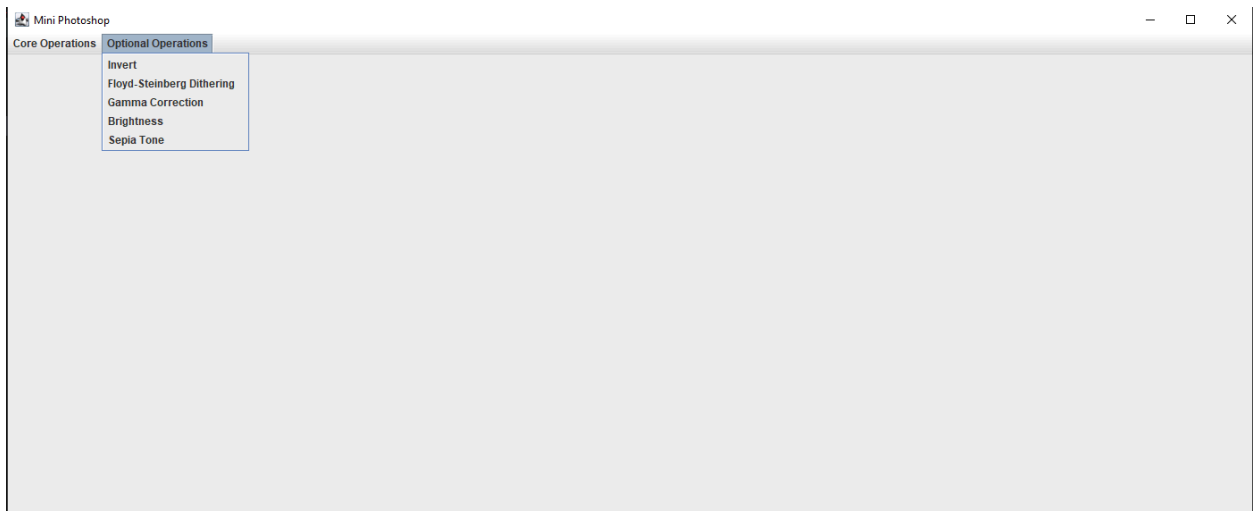
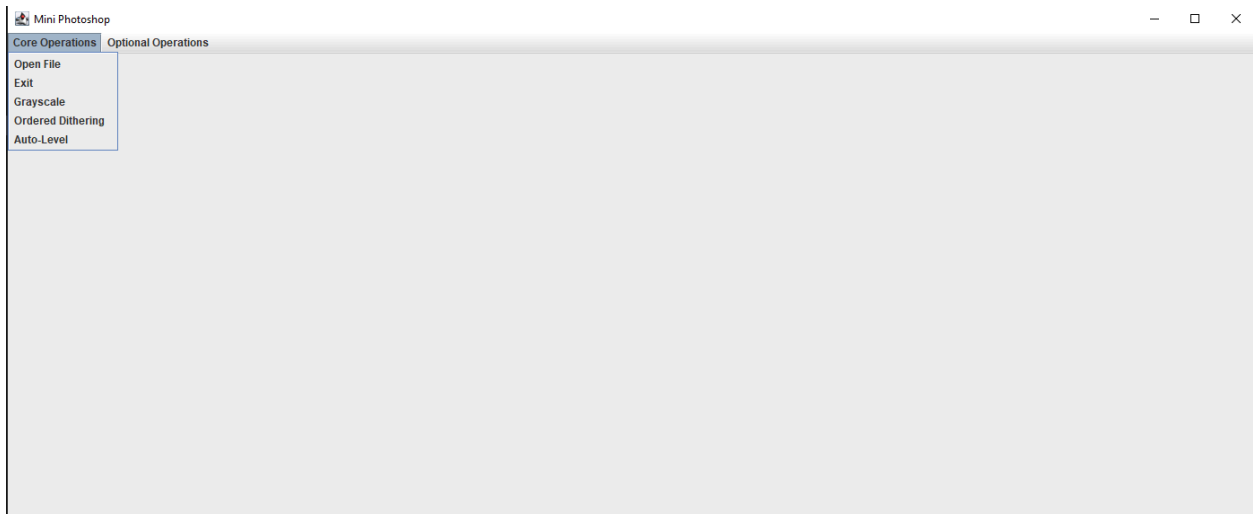
Nicholas Chi (301566158)

For this project, I coded it in Java, utilizing the Java Swing library to create the GUI elements and used the BufferedImage class to handle anything related to manipulating or extracting information about the image.

To create the GUI, I used JFrame to create the window, and JMenuBar to make the bar that will hold the two menus.



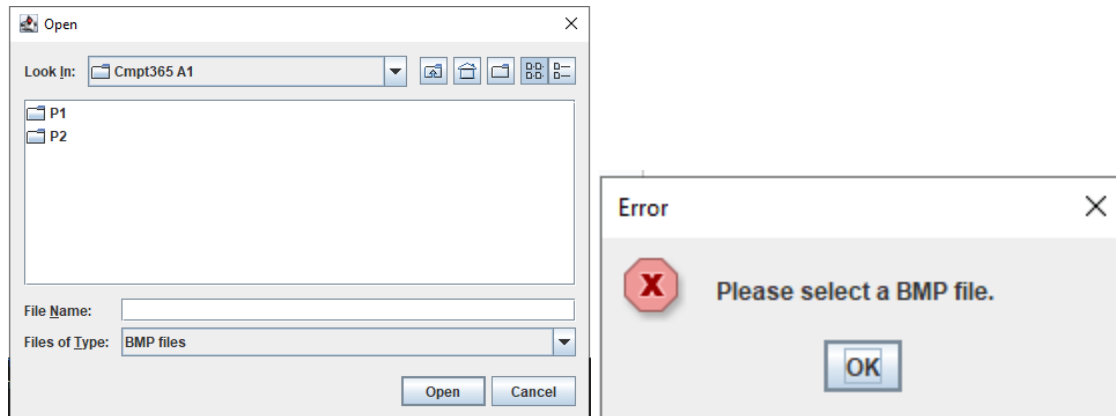
The two menus were created using JMenu and for each menu, I created a JMenuItem that gets added to the respective menu. Each JMenuItem has an ActionListener so that I can know which button is clicked and what to do. To show the images, I store the images in JLabels that get added to a JPanel.



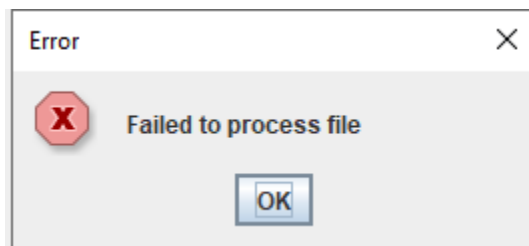
After clicking one of the menu items, the function `actionPerformed` will determine what to do next.

(1) **Open File**

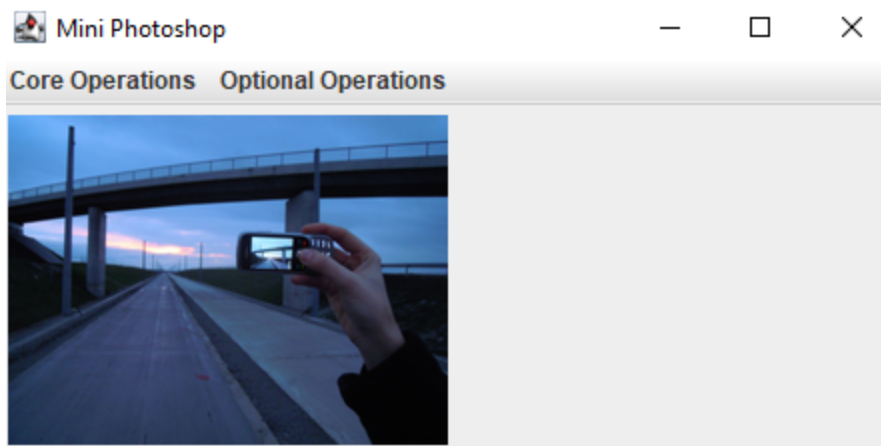
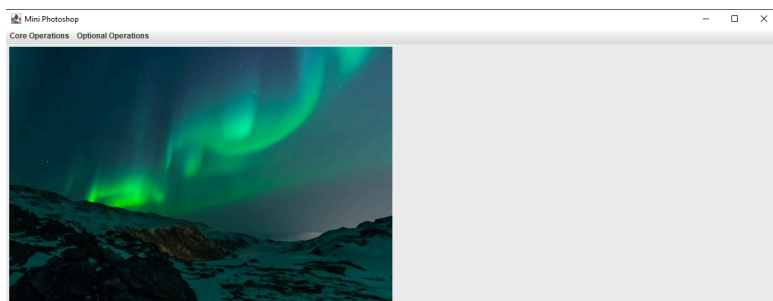
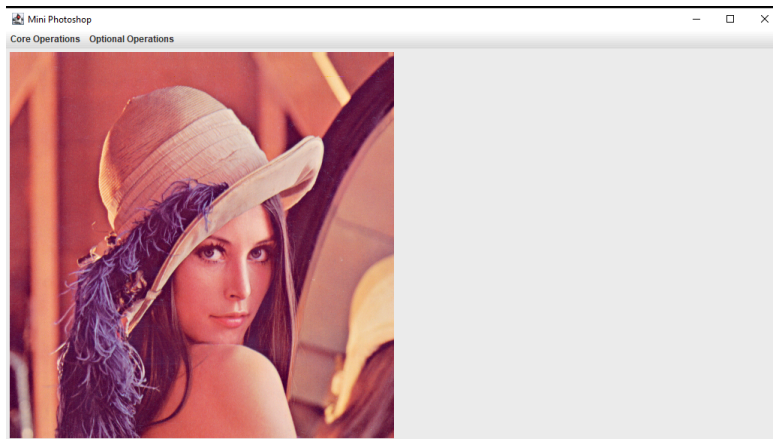
When the user clicks “open file”, it will display to the user a file chooser that was implemented using `JFileChooser`. In our case, we are only interested in BMP files, so I have applied a filter to show such files and will throw an error if a non-BMP file is selected.



Upon choosing a BMP file, I will store the image data in a `BufferedImage` variable called `originalImage` using `ImageIO.read()`. However, in the event there is a problem with the file, it will throw out an error message as well.



`originalLabel.setIcon(new ImageIcon(image))` will convert the `BufferedImage` into an `ImageIcon` and display the image within the label. This line of code will also make sure that if the user wants to load a new BMP file, the previous image will be deleted from the window. The other label `editedLabel` will be initially blank, and that's where we put the edited images.



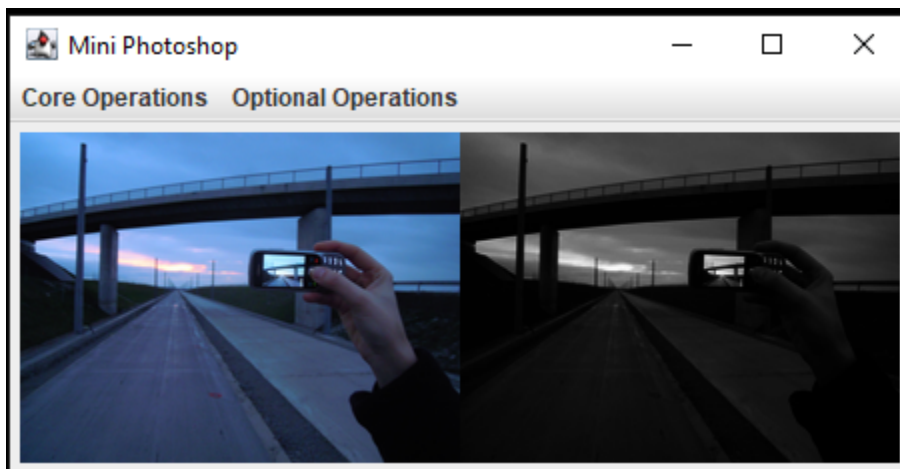
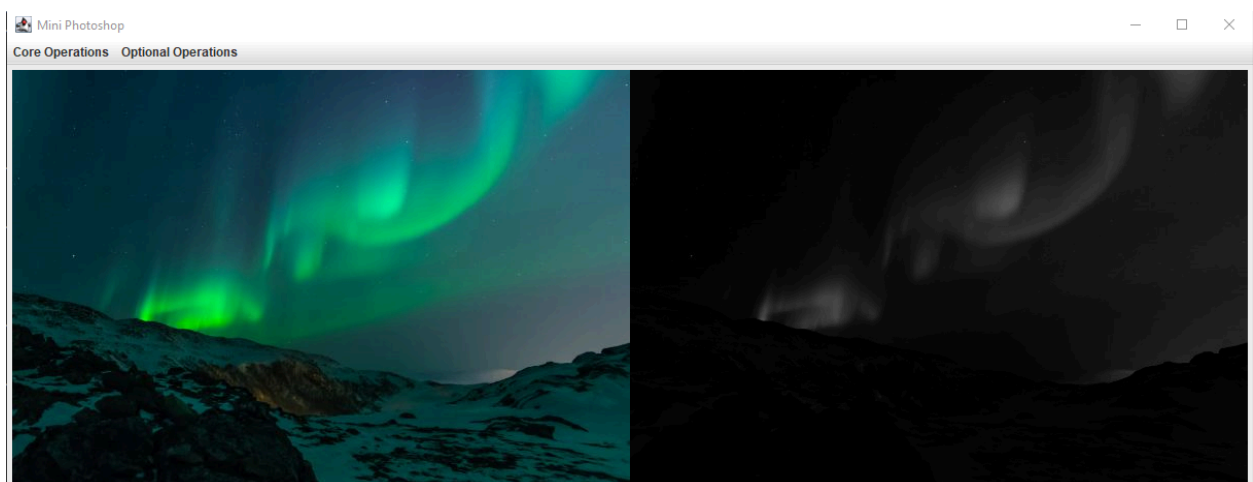
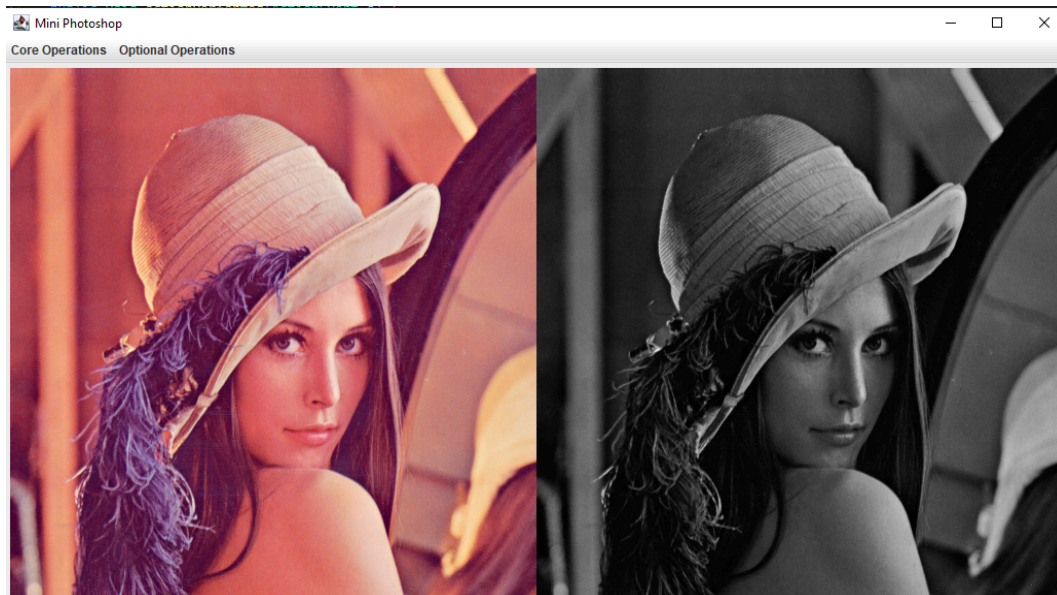
(2) Exit

Clicking the “exit” button will cause the actionPerformed method to terminate the program.

(3) Grayscale

If the user were to click the “Grayscale” button without loading an image it would do nothing. Otherwise, if the user has successfully loaded an image before clicking it, actionPerformed will make a call to a method called convertToGrayscale that will return a grayscaled copy of the image. I store the newly grayscaled image in a BufferedImage variable before passing it into the JLabel editedLabel to display the two images side by side. In the convertToGrayscale method, I create a new BufferedImage variable grayImage with the same dimensions, except that the colours can only be gray. I then iterate through each pixel in the image to find the RGB values

and use the equation $(0.299 * R + 0.587 * G + 0.114 * B)$ to determine the gray value and set the RGB values to that gray value. Then the image is returned.

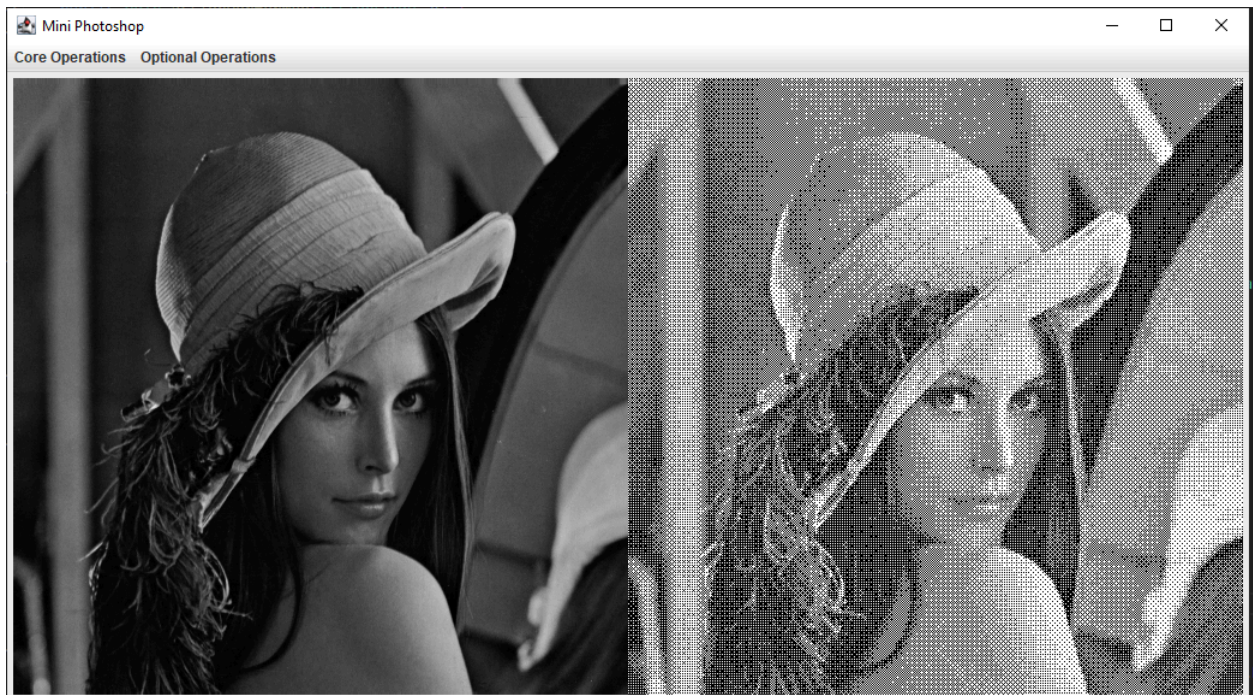


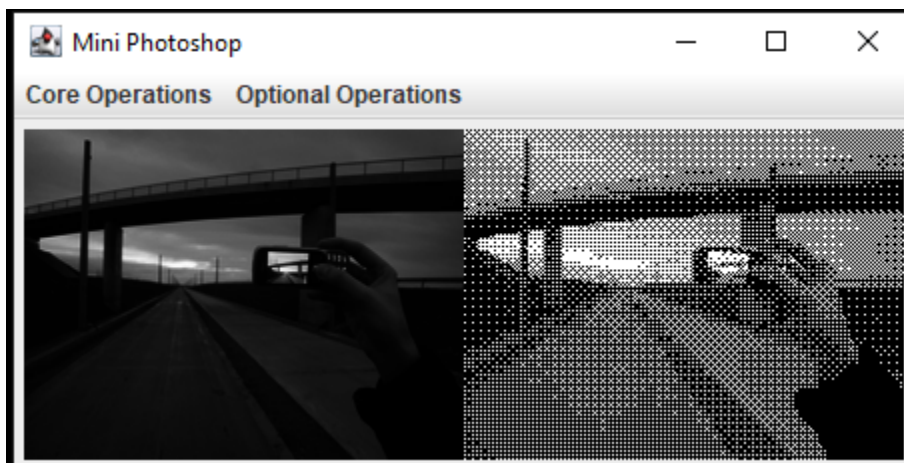
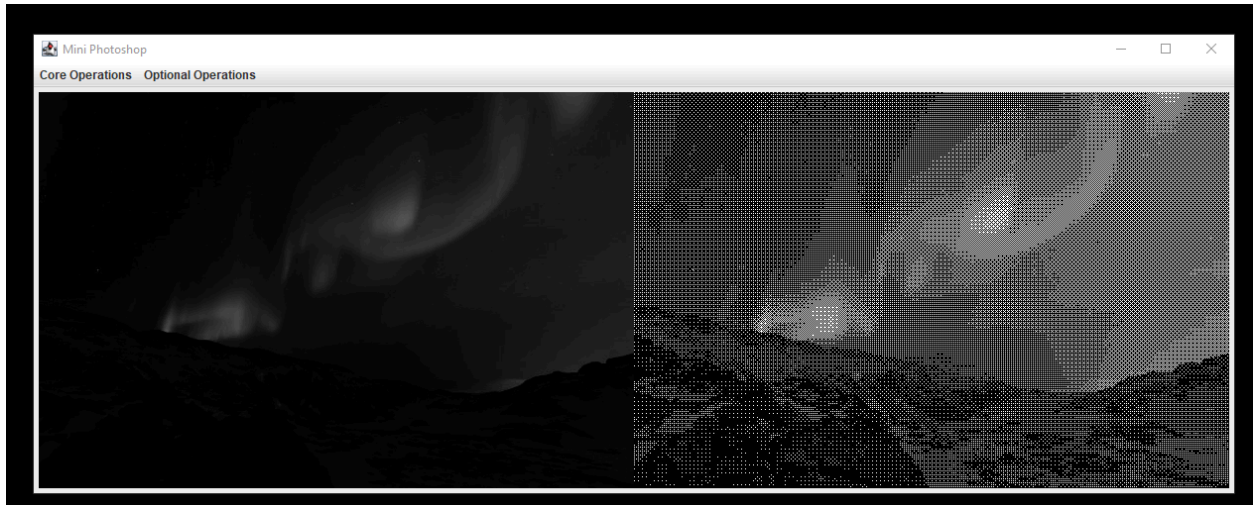
(4) Ordered Dithering

When the user clicks “Ordered Dithering” actionPerformed will first check if an image has been loaded before running ordered dithering. If an image is loaded, then I will call convertToGrayScale to be displayed where the original image was (originalLabel), and orderedDitheringImage to display the image in editedLabel.

In the orderedDitheringImage method, it takes in the original non-edited image. I then create a new BufferedImage variable grayScaleImage that will be a gray image that has ordered dithering applied to it. However, I must first convert the image into a grayscale image using a pixel-by-pixel implementation. I iterated through pixel-by-pixel to find the RGB values and used the equation $(0.299 * R + 0.587 * G + 0.114 * B)$ to determine the gray value and set the RGB values to that gray value. I could have just called my convertToGrayScale method, but I put it inside the orderedDitheringImage to make it easier to look at.

For the dithering matrix, I chose to use a 4x4 Bayer matrix that I got from Wikipedia. However, I scaled all the values beforehand by 255/16 and rounded them. Then I go pixel by pixel and compare the gray value to the matrix. If the gray value is larger than the matrix value, I set the pixel to white, otherwise black. This is applied to every pixel in the BufferedImage grayScaleImage before getting returned.





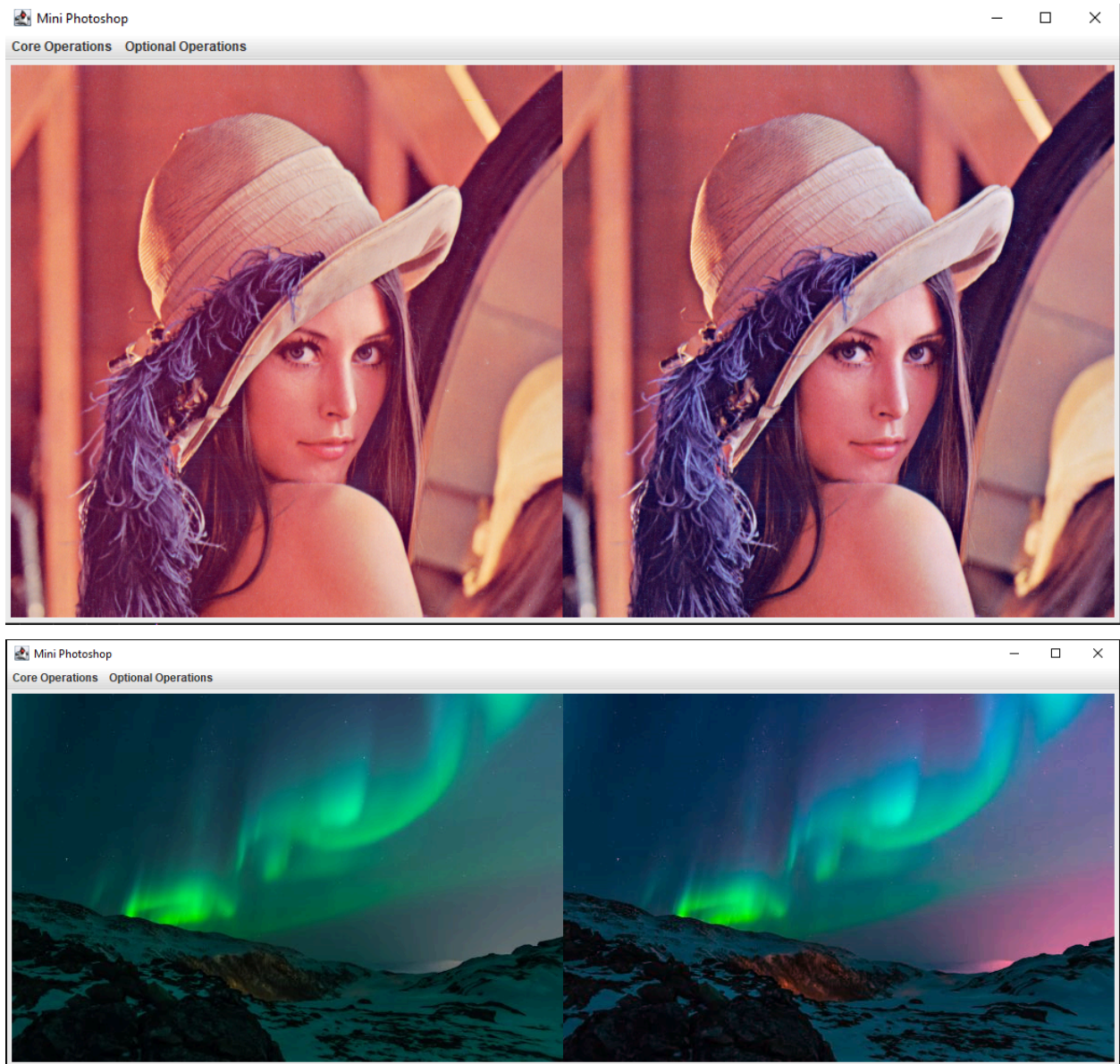
(5) Auto Level

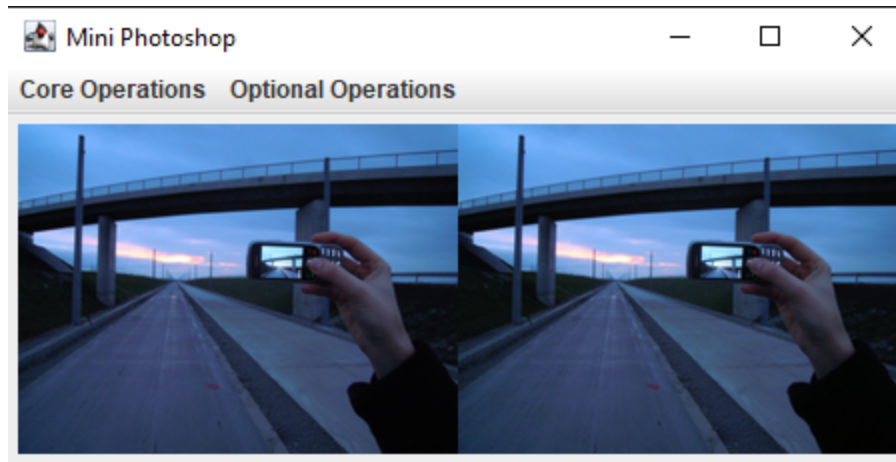
When the user clicks the “Auto-Level” button, the actionPerformed method will first check to see if an image has been loaded. If you were to click the button without loading the image first it would do nothing. I then create a new BufferedImage variable called editImage that will store the auto-leveled image to be passed into the JPanel editedLabel. The original image is passed into the JPanel originalLabel for side-by-side display later.

In the autoLevelImage method, I created a new BufferedImage variable called autoLevelImage which will have the auto-level effect applied to it. I then created variables to pass through the image one time to find the max and min for each RGB value.

Then I pass through the image a second-time pixel-by-pixel and scale the RGB values using the equation $\text{newR} = (r - \text{minR}) * 255 / (\text{maxR} - \text{minR})$. This is done for each respective colour but what it will essentially do is “stretch” the colours. This will mostly benefit the “lower” RGB value by making them larger and therefore brighter, but won’t affect won’t alter as much the

already large values. Then I set each of the RGB values to these newly scaled values and apply it to the BufferedImage variable autoLevelImage before getting returned.



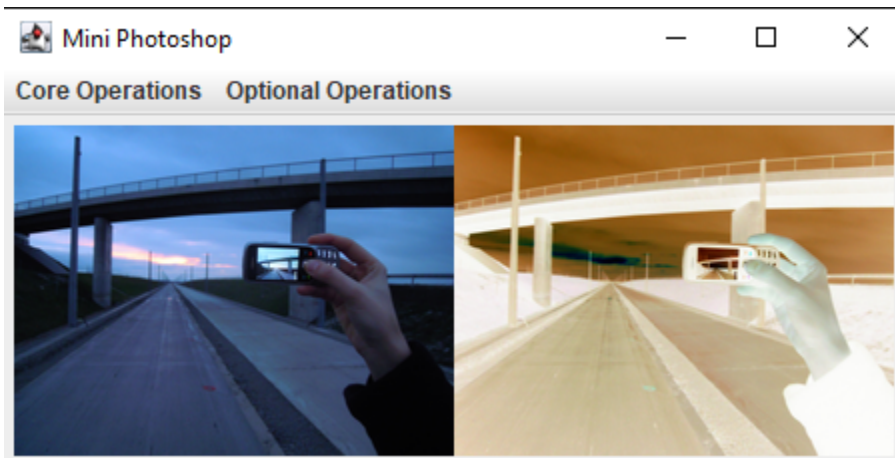
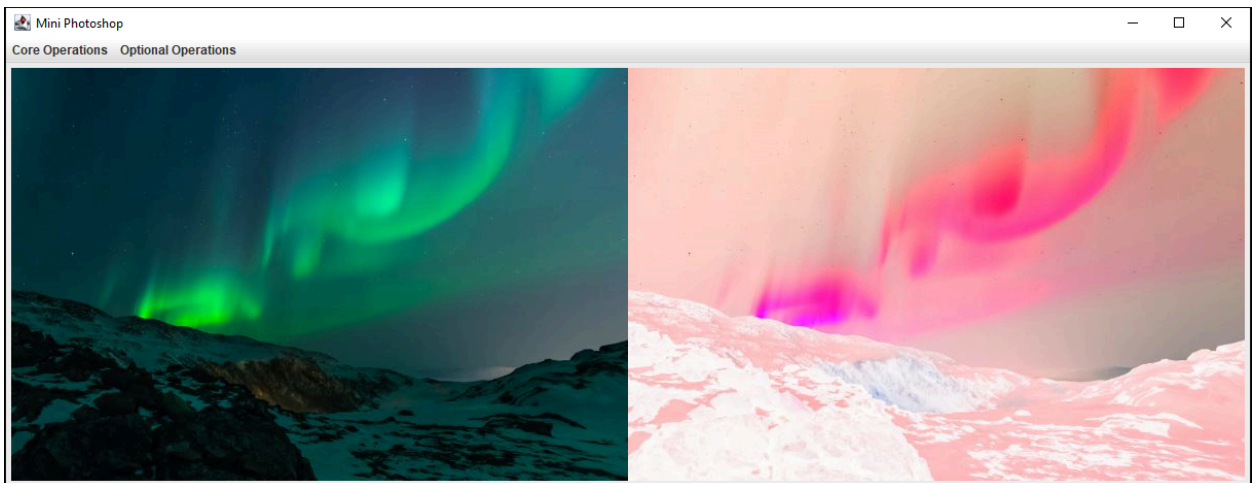
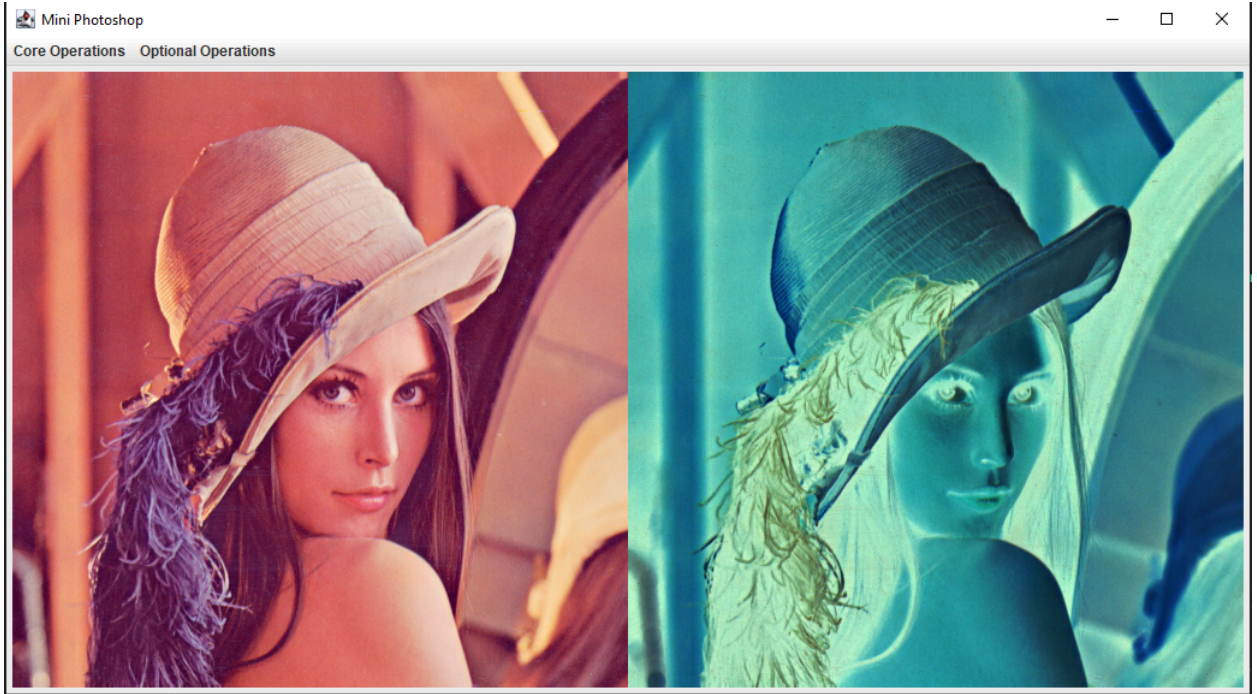


Optional Operations

(1) Invert

When the user clicks the “Invert” button, it will behave similarly to all the other buttons where nothing will happen if an image has not been loaded. If an image has been loaded, then `actionPerformed` will then create a new `BufferedImage` variable called `editImage` which will have the auto-level effect applied to it. `OriginalLabel` will display the original image and `editedLabel` will display the colour inverted image.

In the `autoLevelImage` method, I create a new `BufferedImage` variable called `invertedImage` that will have auto level applied to it. I traverse through each pixel in the original image and extract the RGB values, then perform $255 - \text{RGB}$. This subtraction is done to each colour and is then passed as the new RGB values in `invertedImage` before it gets returned. $255 - \text{the colour}$ will essentially invert the colours by making the original smaller valued colour the dominant colour and the original dominant colour less dominant.



(2) Floyd-Steinberg Dithering

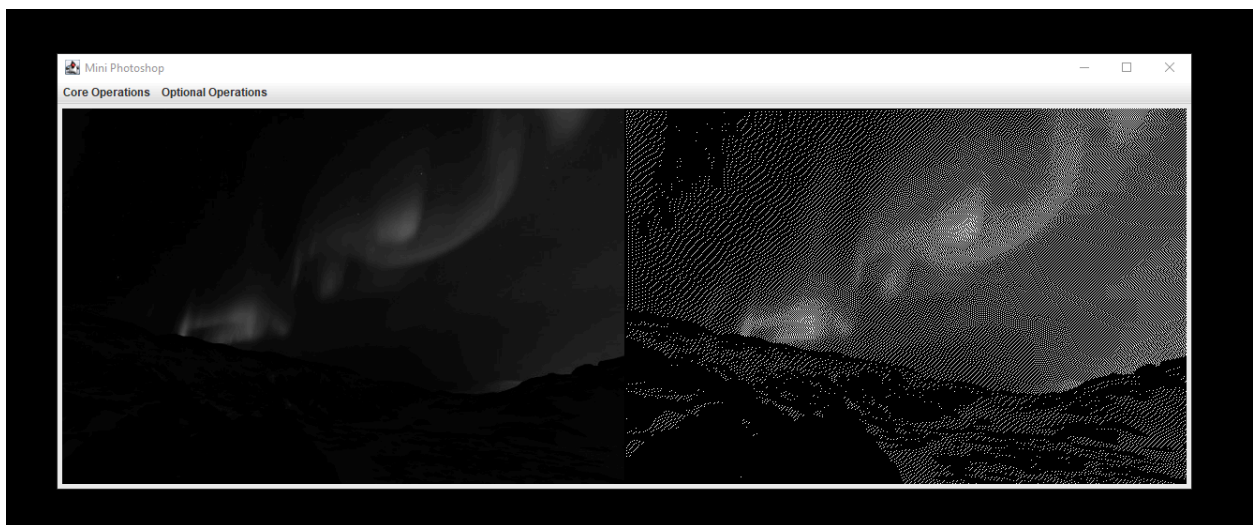
When the user clicks the “Floyd Steinberg Dithering” button, it will behave similarly to all the other buttons where nothing will happen if an image has not been loaded. If an image has been loaded, then `actionPerformed` will then create a new `BufferedImage` variable called `editImage` which will have the Floyd-Steinberg dithering effect applied to it. `OriginalLabel` will display the original image and `editedLabel` will display the dithered image.

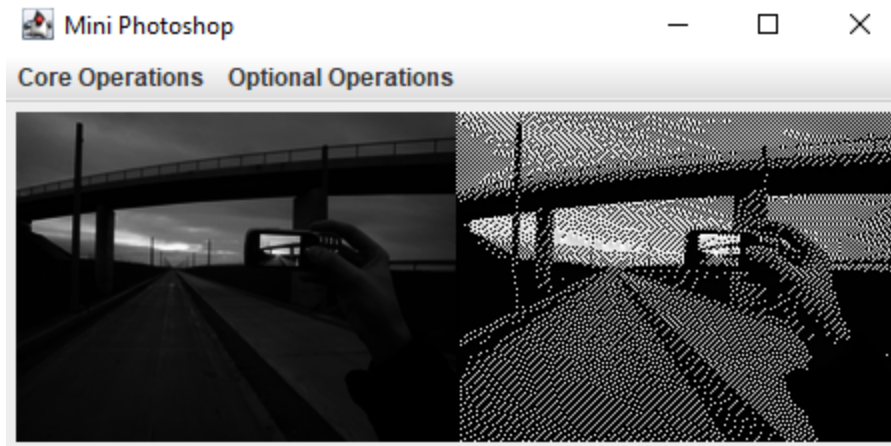
The following was done following this:

https://en.wikipedia.org/wiki/Floyd–Steinberg_dithering

The `floydSeinbergDithering` method creates a new `BufferedImage` variable called `ditheredImage` which will have the effects applied to it. Much like the ordered dithering method, the image is converted to grayscale using the same operation of finding the RGB values and then using the equation $(0.299 * R + 0.587 * G + 0.114 * B)$ to find the gray value and setting the RGB values to that gray value.

Then the image is passed through pixel-by-pixel, and extracts the gray value of the current pixel, calculates the `newPixel` value to either be white or black. Then I calculated the quantization error by subtracting the `oldPixel` with the `newPixel`. The `DiteredImage` variable's RGB values are then set to be the `newPixel` values. The next few lines deal with the error diffusion and handles the quantization error in another method called `distributeError`.



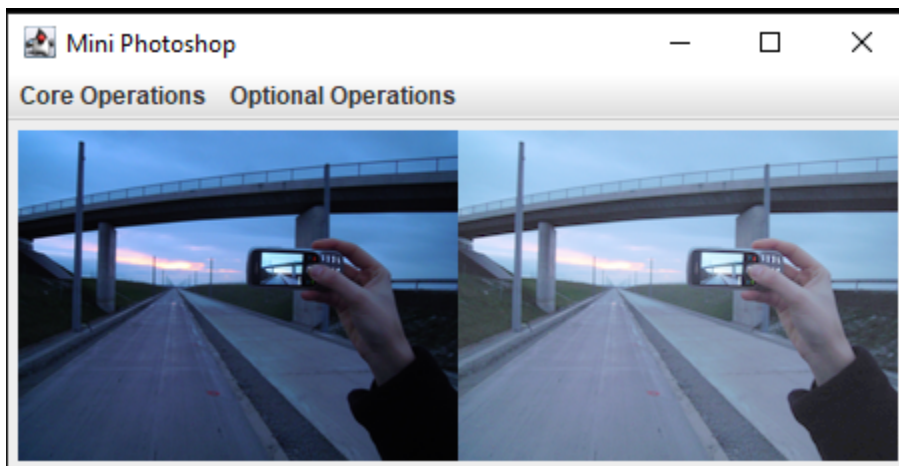
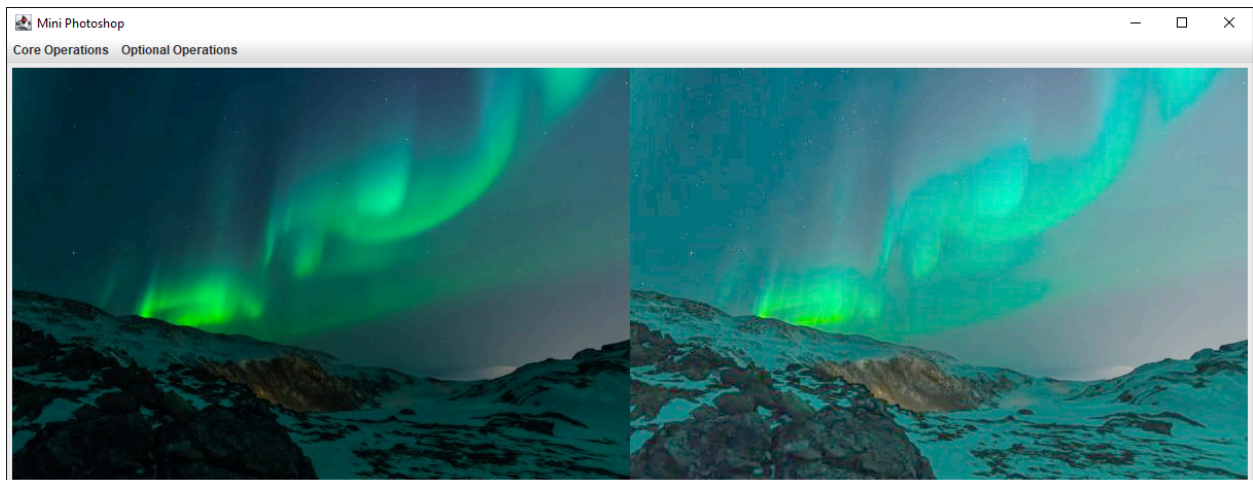
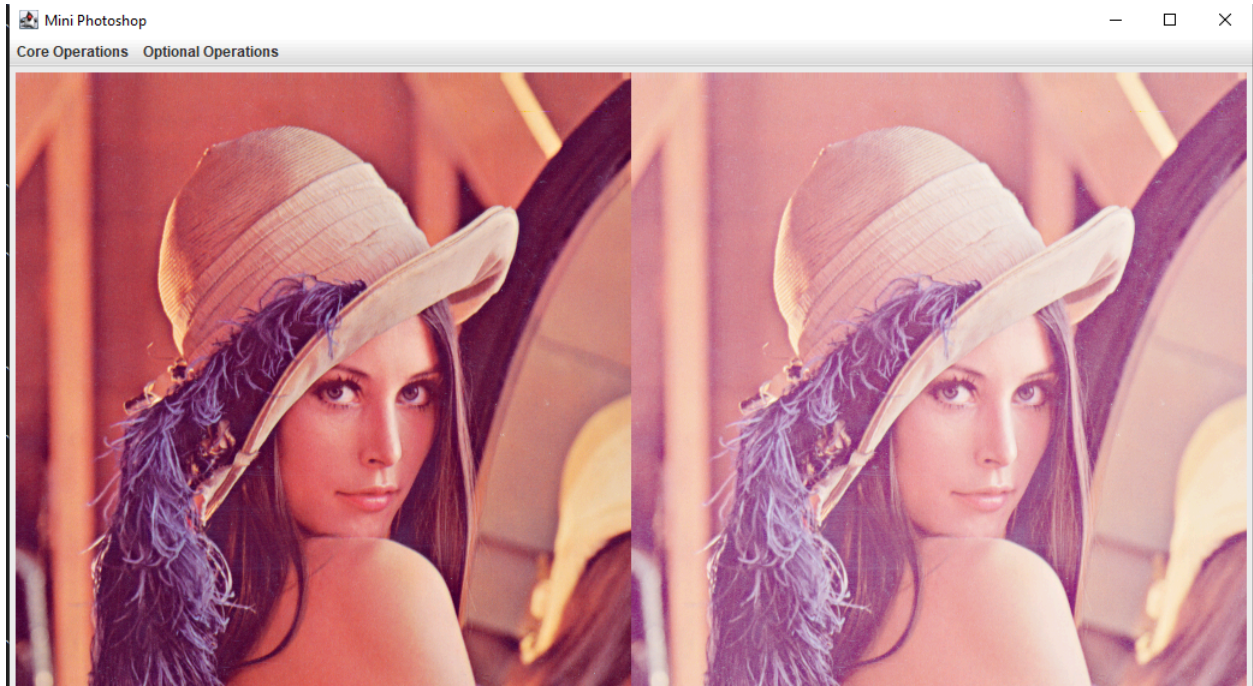


(3) Gamma Correction

When the user clicks the “Gamma Correction” button, it will behave similarly to all the other buttons where nothing will happen if an image has not been loaded. If an image has been loaded, then `actionPerformed` will then create a new `BufferedImage` variable called `editImage` which will have the Gamma Correction applied to it. `OriginalLabel` will display the original image and `editedLabel` will display the gamma corrected image. It will also prompt the user to enter a gamma value to use for gamma correction.

In the `gammaCorrection` method, a new `BufferedImage` variable called `gammaCorrectedImage` is created which will be the image with gamma correction applied to it. I also calculate the `gammaValue` that will be used in the calculation later.

I used a for loop to go through pixel-by-pixel to apply gamma correction with the gamma correction equation ($255 * \text{Math.pow}(r / 255.0, \text{gammaValue})$), this operation is applied to each colour respectively. I check the RGB colour to make sure that it is not greater than 255 before setting the RGB of the `BufferedImage` `gammaCorrectedImage` to these gamma-corrected RGB values.

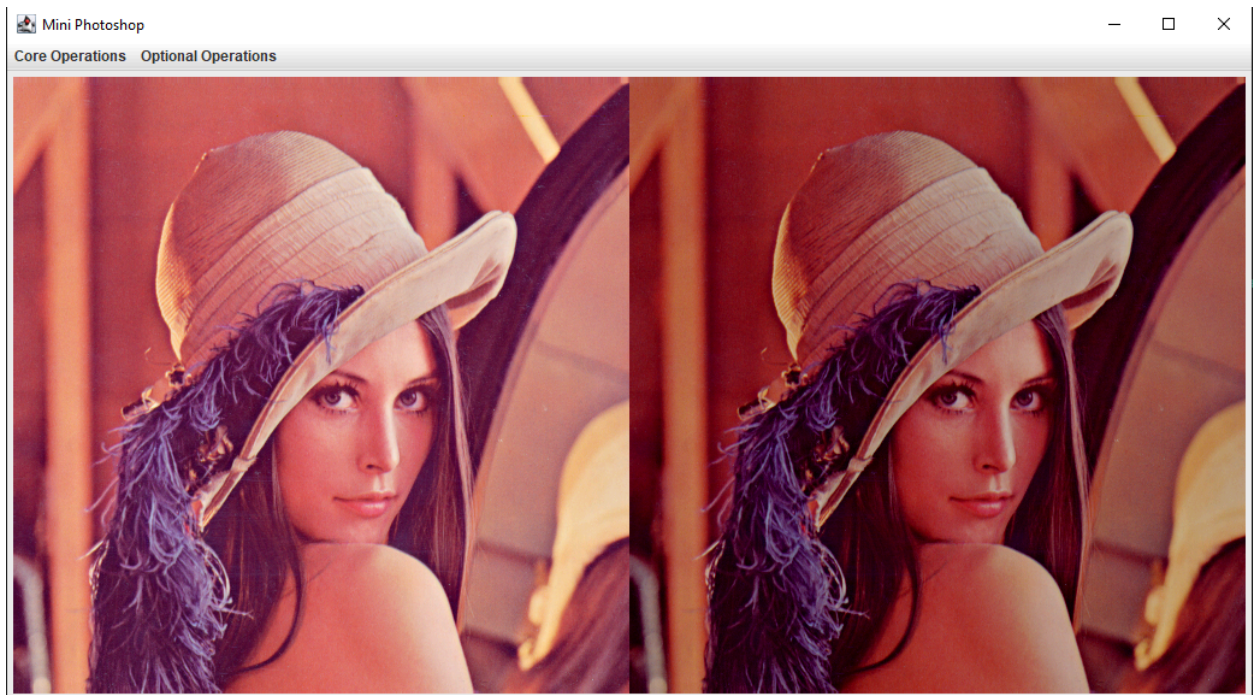


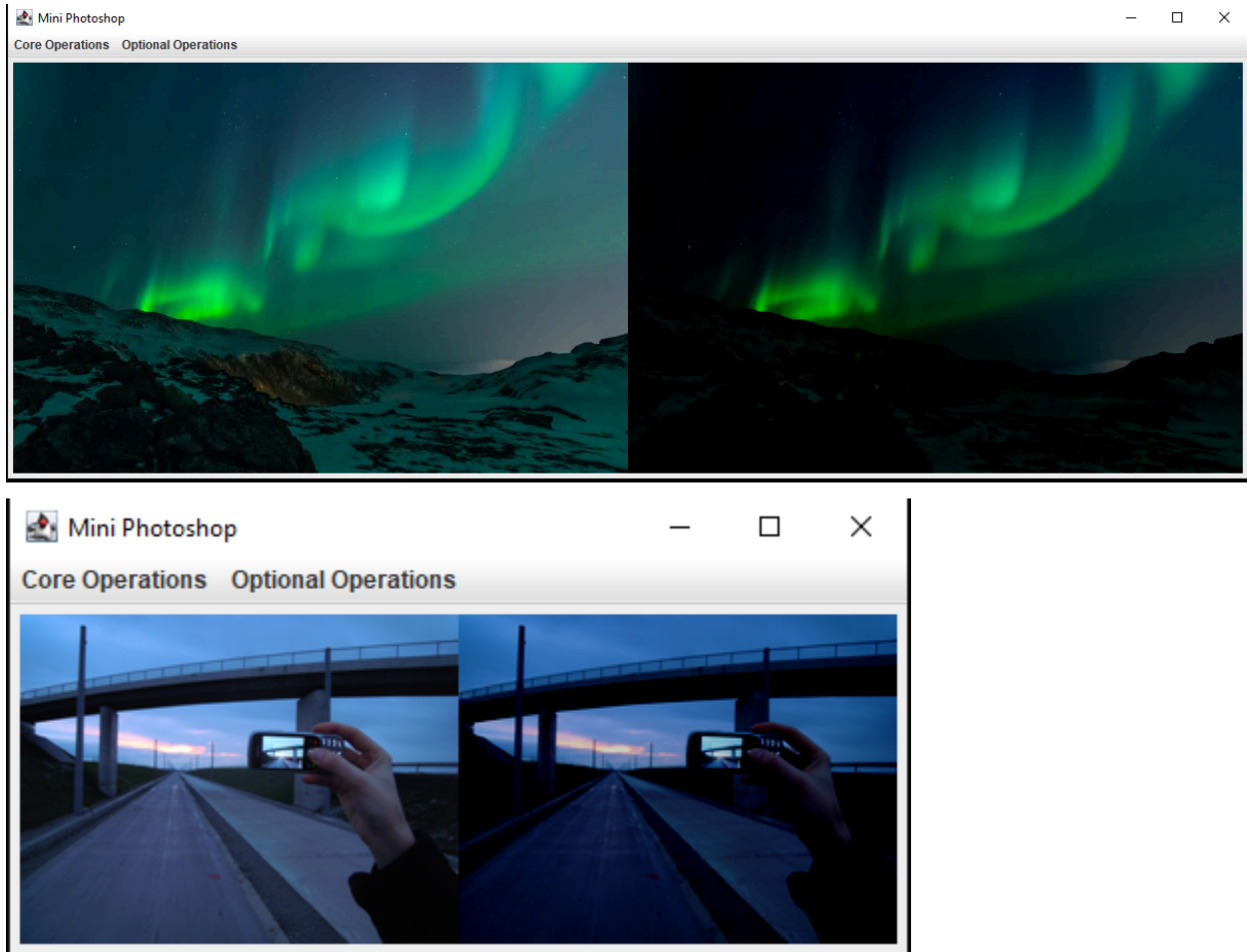
(4) Brightness Adjustment

When the user clicks the “Brightness” button, it will behave similarly to all the other buttons where nothing will happen if an image has not been loaded. If an image has been loaded, then actionPerformed will then create a new BufferedImage variable called editImage which will have the brightness effect applied to it. OriginalLabel will display the original image and editedLabel will display the brightened/darkened image. It will also prompt the user to enter a brightness value, either positive to make the image brighter or negative to make the image darker.

In the brightness method, a new BufferedImage variable called brightenedImage is created which will be the image that has its brightness adjusted to whatever the user chooses.

I used a for loop to go through pixel-by-pixel in the image to extract the RGB values and apply the brightness value to each RGB value. The Math.min and Math.max make sure that the values are between 0-255. These adjusted RGB values are then set to be the RGB values of the brightenImage before it gets returned (see below where I used a value of -50 to make the images darker).





(5) Sepia Filter

When the user clicks the “Sepia Tone” button, it will behave similarly to all the other buttons where nothing will happen if an image has not been loaded. If an image has been loaded, then `actionPerformed` will then create a new `BufferedImage` variable called `editImage` which will have the brightness effect applied to it. `OriginalLabel` will display the original image and `editedLabel` will display the image with a sepia filter applied to it.

`sepiaTone` for RGB values was taken from this:

<https://www.geeksforgeeks.org/image-processing-in-java-colored-image-to-sepia-image-conversion/>

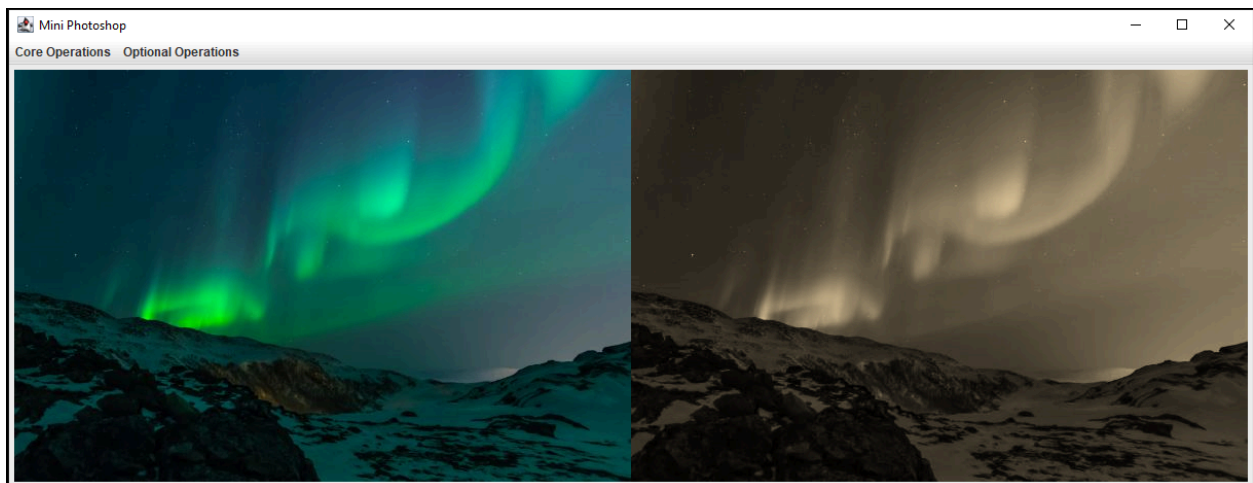
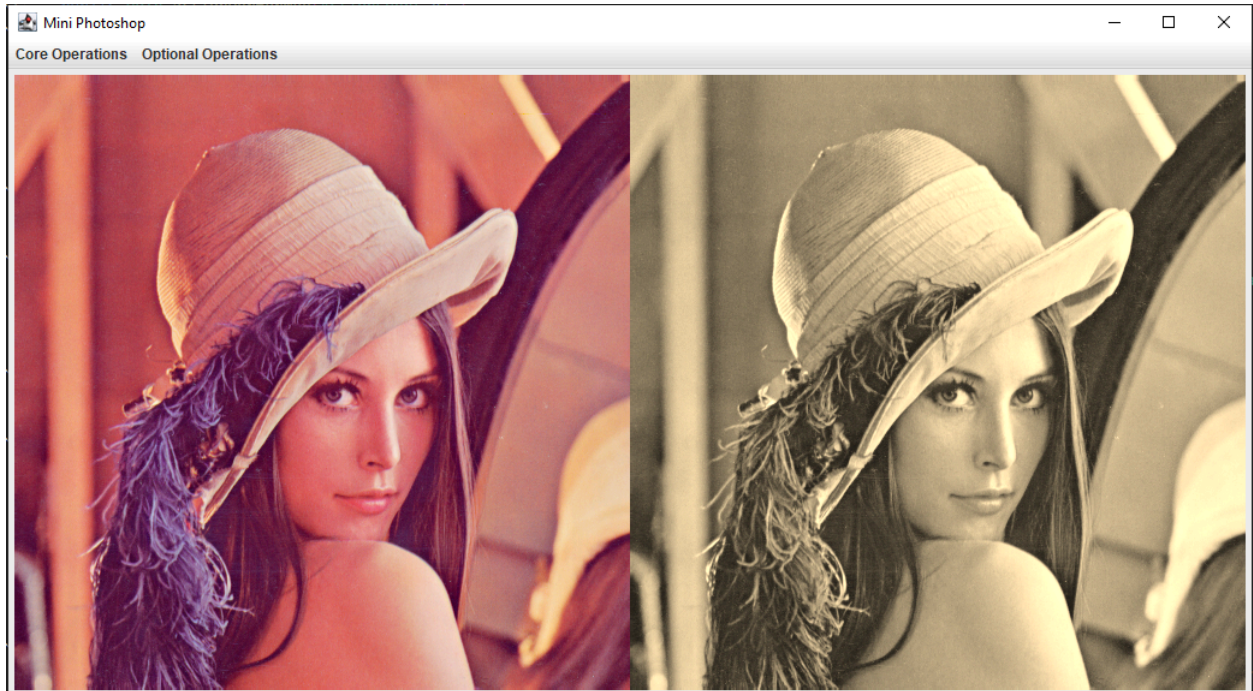
In the `sepiaTone` method, I create a new `BufferedImage` variable called `sepiaImage` which will have the `sepiaTone` altered RGB values applied to it.

I used a for loop to go through pixel-by-pixel in the image to extract the RGB values and apply the following calculations:

```
int newRed = (int) (0.393 * r + 0.769 * g + 0.189 * b);
```

```
int newGreen = (int) (0.349 * r + 0.686 * g + 0.168 * b);  
int newBlue = (int) (0.272 * r + 0.534 * g + 0.131 * b);
```

These adjusted RGB values are then set to be the RGB values of the sepiaImage before it gets returned.





Mini Photoshop



Core Operations Optional Operations

